BUILD A VECTOR DATABASE AND PERFORM SIMILARITY-BASED DOCUMENT RETRIEVAL

Aim

To create a vector database using FAISS, store document embeddings in a vector index, and retrieve the most relevant documents for a user query using cosine similarity.

Software Requirements

Python 3.x, NumPy, scikit-learn, and FAISS. Install the required packages using: `pip install faiss-cpu scikit-learn numpy`. An IDE such as VS Code can be used.

Procedure

1. Create a Python file named faiss_vector_database.py.

2. Import NumPy, TfidfVectorizer, and the normalization utility. Import FAISS when it is installed.

3. Prepare a collection of text documents to act as the document database.

4. Fit the TF-IDF vectorizer on the documents and convert each document into a numerical vector.

5. Convert the user's query into a vector using the same vectorizer.

6. Normalize document and query vectors to unit length. This makes their inner product equivalent to cosine similarity.

7. Create a FAISS IndexFlatIP index using the embedding dimension and add all document vectors to the index.

8. Search the index with the query vector and request the top-k results.

9. Display each retrieved document with its ID and similarity score. If FAISS is not installed, the provided script runs a NumPy cosine-search fallback.

Program Code

```
"""
Build a vector database with FAISS and retrieve similar documents.
Install: pip install faiss-cpu scikit-learn numpy
If FAISS is not installed, this script uses an exact cosine-search
fallback so the workflow can still be demonstrated.
"""
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import normalize

documents = [
    "Machine learning helps computers learn patterns from data.",
    "Deep learning uses neural networks with many layers.",
    "A database stores and organizes information for retrieval.",
    "Renewable energy comes from sources such as sunlight and wind.",
    "Natural language processing enables computers to understand text.",
    "FAISS is a library for efficient similarity search over dense vectors."
]

query = "How can a computer find information about learning from data?"

# 1. Convert documents to numerical vectors (embeddings).
vectorizer = TfidfVectorizer(stop_words="english")
document_vectors = vectorizer.fit_transform(documents).toarray().astype("float32")
query_vector = vectorizer.transform([query]).toarray().astype("float32")

# 2. Normalize vectors so inner product equals cosine similarity.
document_vectors = normalize(document_vectors, norm="l2").astype("float32")
query_vector = normalize(query_vector, norm="l2").astype("float32")

# 3. Build a FAISS vector index and add document vectors.
try:
    import faiss
    dimension = document_vectors.shape[1]
    index = faiss.IndexFlatIP(dimension)  # exact inner-product search
    index.add(document_vectors)
    scores, indices = index.search(query_vector, k=3)
    engine = "FAISS IndexFlatIP"
except ImportError:
    # Fallback for environments where faiss-cpu is not installed.
    # It computes the same normalized inner-product/cosine ranking.
    similarities = query_vector @ document_vectors.T
```

```
        indices = np.argsort(similarities[0])[::-1][:3].reshape(1, -1)
        scores = np.take_along_axis(similarities, indices, axis=1)
        engine = "NumPy cosine-search fallback (install faiss-cpu for FAISS)"

    # 4. Display top-k retrieved documents.
    print("VECTOR DATABASE: DOCUMENT RETRIEVAL")
    print("Search engine:", engine)
    print("Query:", query)
    print("\\nTop 3 Retrieved Documents:")
    for rank, (doc_id, score) in enumerate(zip(indices[0], scores[0]), start=1):
        print(f"\\nRank {rank} | Document ID: {doc_id} | Similarity: {float(score):.4f}")
        print("Text:", documents[int(doc_id)])
```

Execution Screenshot

Captured terminal output from running the included Python program:

Python Terminal | Vector Database Retrieval

```
VECTOR DATABASE: DOCUMENT RETRIEVAL
Search engine: NumPy cosine-search fallback (install faiss-cpu for FAISS)
Query: How can a computer find information about learning from data?
\nTop 3 Retrieved Documents:
\nRank 1 | Document ID: 0 | Similarity: 0.4061
Text: Machine learning helps computers learn patterns from data.
\nRank 2 | Document ID: 2 | Similarity: 0.2736
Text: A database stores and organizes information for retrieval.
\nRank 3 | Document ID: 1 | Similarity: 0.1727
Text: Deep learning uses neural networks with many layers.
Spreadsheet runtime warmup failed during python startup
Traceback (most recent call last):
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/patches/warm_spreadsheet_runtime_on_startup.py", line 26,
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py", line 785, in warm_spreadsheet_run
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py", line 718, in _warm_feature_flows
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py", line 509, in _warm_drawing_and_in
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/generated/interface/models.py", line 32861, in bring_to_t
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/remote.py", line 764, in __call__
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/client.py", line 175, in call
artifact_tool.rpc.client.RemoteError: Element ordering requires an owning element collection.

[Process finished with exit code 0]
```

Result

The program executed successfully. It converted the sample documents and query into normalized vectors, created the similarity-search workflow, and retrieved the top three matching documents with similarity scores. In this environment FAISS was not installed, so the screenshot shows the NumPy fallback, which uses the same normalized inner-product calculation. To run the actual FAISS index, install faiss-cpu and rerun the same script.

Note: TF-IDF is used here to create a simple vector representation so the experiment can run without downloading a language model. For stronger semantic retrieval, replace TF-IDF with sentence-transformers embeddings, then normalize those vectors and add them to the same FAISS index.